Eduardo Silva Sinico – 0444/25-1 & Gabriel Barbosa Favaro – 0301/25-1 • Computação em Nuvem e DevOps

Segundo encontro de DevOps (09/11/2026)

Tópicos:

- Identificação da dupla 11
- Relatório da dupla: Parte 1 e Parte 2
- Questionário

Identificação da dupla 11:

- Eduardo Silva Sinico (Aluno A) & Gabriel Barbosa Favaro (Aluno B)
- Usuários do Github: Edu1Sinico e GabrielFavaroo
- Repositório do Github: https://github.com/Edu1Sinico/projeto-devops-dupla-11

Relatório da dupla – Parte 1:

Aluno A:

1. O que eu fiz hoje. Cite os arquivos que criei ou alterei, os comandos que rodei e o meu papel (aluno A ou B).

O meu papel hoje foi realizar os passos do aluno A, conforme dito anteriormente no arquivo: passo 2, passo 3, passo 4 e passo 7.

Passo 2 - Realizei a criação dos arquivos app/requirements.txt, app/app.py e app/test_app.py.

Passo 3 - Executei os seguintes comandos:

```
cd app

python3 -m venv .venv

source .venv/bin/activate

pip install -r requirements.txt

python app.py
```

Passo 4 - Realizei o commit dos dados criados nos passos anteriores:

```
cd ..

git add app .gitignore

git commit -m "API Flask da dupla"

git push
```

Passo 7 - Por fim, executei os seguintes comandos do Docker para testes:

docker build -t projeto-devops:v2 .

docker rm -f api

docker run -d --name api -p 5000:5000 projeto-devops:v2

curl http://localhost:5000/aluno

Por fim, elaborarei as anotações das previsões mencionadas nos passos.

2. O que eu aprendi. Explique com as suas palavras o que é um Dockerfile, uma imagem e um contêiner.

O DockerFile é o arquivo responsável pela configuração do contêiner do Docker, esse processo é feito através da "imagem", que é um modelo de como o contêiner deve ser estruturado.

3. Do Dockerfile ao contêiner. Descreva, na ordem, o que acontece desde o arquivo Dockerfile até a API responder no curl.

No Dockerfile, nós definimos toda estrutura (modelo) do contêiner, toda instrução do que deve ser feito é dado no Dockerfile. Após aplicarmos os comandos do Docker, nós montamos o contêiner, após isso, conseguimos rodá-lo e executar o comando "curl" para acessar nossa aplicação.

4. Contêiner ou máquina virtual. Qual é a diferença? Cite pelo menos um número que você mediu no passo 8.

A máquina virtual é responsável por emular um sistema operacional, utilizando da parte dos recursos físicos de nosso computador para toda a operação. Já o contêiner, ele não possui um sistema operacional próprio, mas compartilha diretamente do kernel, no qual, utiliza dos recursos para isolar a aplicação e suas dependências.

Há também a diferença de tamanho e velocidade, no qual, a VM costuma utilizar vários GB e possui um tempo de dezenas de segundas a minutos. O contêiner apresenta ser menor no espaço e tempo, como no nosso exemplo do projeto, nosso contêiner possui 52.7MB e um tempo de inicialização de 0.556.

5. O que deu errado ou foi difícil, e como a dupla resolveu.

No momento, não enfrentei dificuldades durante o desenvolvimento deste segundo encontro.

6. Uma dúvida que ainda tenho.

Não apresento dúvidas no desenvolvimento deste segundo encontro.

Extra (Aluno A): No Dockerfile, por que a lista de dependências é copiada e instalada antes de copiar o código da API? O que isso mudou no build do passo 7?

A lista de dependências é copiada e instalada antes do código da API para aproveitar o cache de camadas do Docker, evitando a reinstalação desnecessária das bibliotecas quando apenas o código da aplicação é alterado.

No passo 7, ao reconstruir a imagem, observei que as etapas de cópia do requirements.txt e instalação das dependências apareceram como CACHED, demonstrando que o Docker reutilizou os resultados anteriores.

No meu teste, a etapa de cópia do código também apareceu como CACHED, indicando que o Docker já possuía um resultado reutilizável para essa etapa.

Aluno B:

1. O que eu fiz hoje. Cite os arquivos que criei ou alterei, os comandos que rodei e o meu papel (aluno A ou B).

Meu trabalho como Aluno B foi voltado a gerenciar o ambiente Docker e suas respectivas configurações, como demonstrado nos passos a baixo:

Passo 3 - Realizei o Git Pull para receber as alterações feitas por meu colega, partindo assim para os próximos passos referentes aos arquivos Docker.

Passo 5 - Montei do Dockerfile, também chamado de Docker image com os requisitos do ambiente, seguindo este modelo:

```
FROM python:3.12-slim

WORKDIR /app

COPY app/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

EXPOSE 5000 CMD ["python", "app.py"]
```

Também o arquivo dockerignore para evitar que arquivos indesejados cheguem na imagem final, gerando peso e processamento desnecessário. O dockerignore segue este modelo:

```
 **/.venv

 **/__pycache__

**/.pytest_cache

 .git

 .github site
```

Por fim, adicionei os arquivos Dockerfile e dockerignore ao repositório local via git Add, realizei um commit para arquivar as alterações com um contexto no título e enviei todas as alterações ao repositório remoto através do git push.

Passo 9 - Após gerar algumas alterações de testes, as removi ao substitui-las pelo último estado desejado da aplicação que ocorreu pelo meu **git push no passo 5**. Também conferi a situação do repositório através do comando:

```
git log --oneline -5
```

2. O que eu aprendi. Explique com as suas palavras o que é um Dockerfile, uma imagem e um contêiner.

O Dockerfile é o arquivo que contém uma lista de passos consecutivos necessários para que a aplicação possa funcionar, isso inclui, instalar o ambiente da máquina virtual e dependências de software e middleware, compilar o código e executar o código compilado.

O Dockerfile define as regras, aos termos um Dockerfile temos necessariamente uma imagem que reúne todos esses passos. Para ter acesso a bibliotecas, sistemas operacionais, etc. Existem imagens destinadas a esse

oficio, as quais mencionamos e utilizamos na nossa aplicação. O container é a instancia da imagem, sendo um ambiente virtual isolado que segue os requisitos impostos na imagem.

3. Do Dockerfile ao contêiner. Descreva, na ordem, o que acontece desde o arquivo Dockerfile até a API responder no curl.

Ao realizar a execução de um Dockerfile, será criado um ambiente virtual seguindo os padrões impostos na imagem, e nele será instalado quaisquer softwares ou middleware, seguindo depois para o processo de obtenção da imagem do código da aplicação, que será compilado e executado nesse ambiente container.

Para que o container possa estar exposto a requisições, este deve ter na imagem do dockefile a porta designada para a aplicação. Por meio desta, a aplicação pode receber e processar requisições http, sendo o Curl, uma das formas de gerar requisições http através do terminal.

4. Contêiner ou máquina virtual. Qual é a diferença? Cite pelo menos um número que você mediu no passo 8.

Através da análise de tamanho (Size), percebemos que o container é uma escolha muito mais leve do que uma máquina virtual.

5. O que deu errado ou foi difícil, e como a dupla resolveu.

Ao tentar iniciar o container sem atribuir uma porta para requisições, ele não foi capaz de atribuir uma porta sozinho, portanto, a aplicação ficou momentaneamente inalcançável, até que adicionamos a respectiva porta e foi possível realizar as requisições.

6. Uma dúvida que ainda tenho.

Existe alguma vantagem em utilizar o Linux WSL em comparação a ambientes de médio porte como Linux Ubuntu ou Alpine?

Extra (Aluno B): Repetir o passo 3 do zero deu trabalho. Liste o que você precisou fazer e o que deu errado, e diga o que o contêiner eliminaria dessa lista.

Precisei executar a aplicação sem ter um container. Para isso, foi instalado na máquina os requisitos da API.

Com um container esse processo seria executado em um ambiente isolado e não na máquina real, aonde ele estaria isolado e poderia repetir qualquer passo com muito mais facilidade, permitindo alterações e melhorias baseadas no comportamento do container e sua imagem.

Ao utilizar uma máquina física, o código se torna repetitivo e sujeito a diversos erros e divergências.

Relatório da dupla – Parte 2:

1. Previsões e resultados dos passos 3, 6 e 7, no quadro abaixo.

Passo	O que prevemos	O que aconteceu

3. Rotas /saude e /aluno	Que cada rota retornasse os campos esperados no formato JSON na porta da aplicação.	Ao designar uma porta e executar as requisições via curl, houve o retorno correto em formato JSON como o esperado.
6. Contêiner sem mapear a porta	Que o próprio Docker atribuiria uma porta padrão para o container, por exemplo 3456 e subiria normalmente.	Ao tentar executar um curl na porta, este foi o retorno curl: (7) Failed to connect to localhost port 5000 after 0 ms: Couldn't connect to server Nisso é perceptível que não é possível realizar requisições sem atribuir uma porta antes no processo.
7. Linhas refeitas ao mudar só a MENSAGEM	Somente algumas camadas seriam refeitas, as camadas mais inferiores referentes ao ambiente permaneceriam cacheadas.	As seguintes camadas não foram refeitas pois não havendo alterações, foi utilizado o cache, sendo: => CACHED [2/5] WORKDIR /app => CACHED [3/5] COPY app/requirements.txt . => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt

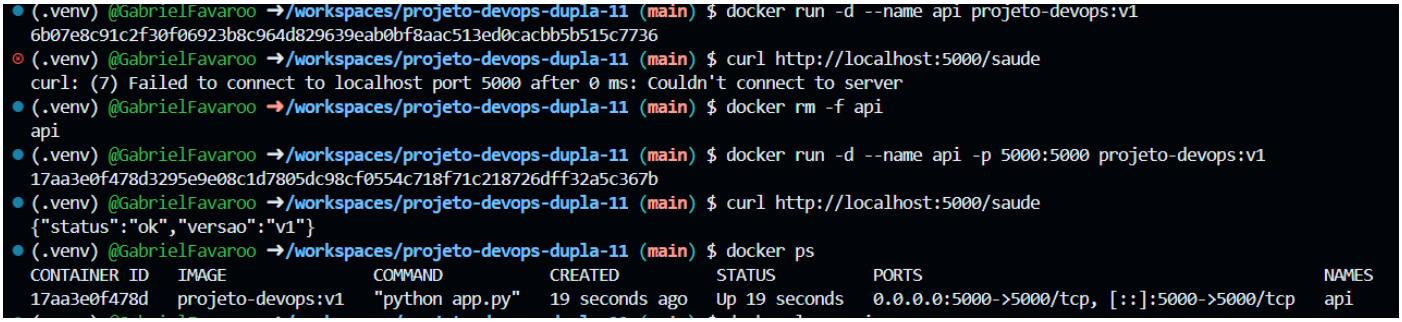
2. Quadro contêiner x máquina virtual do passo 8, preenchido

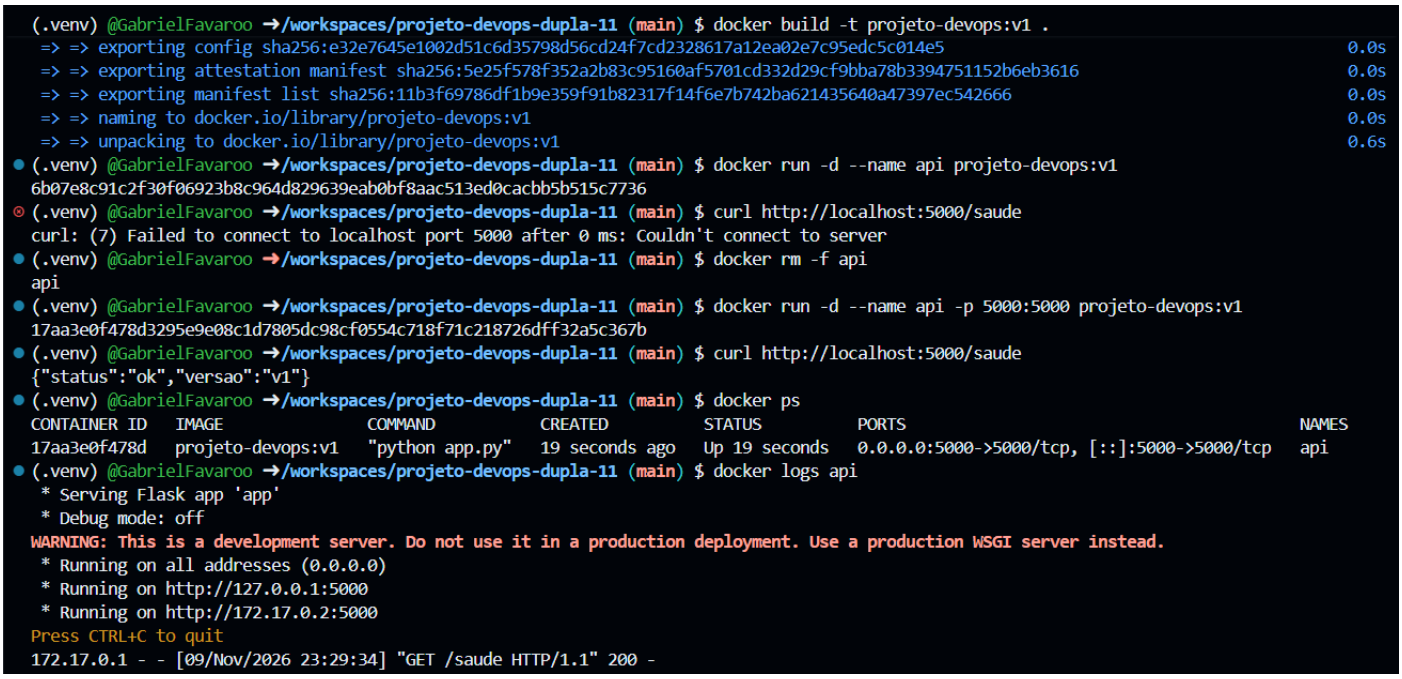
Característica	Máquina virtual	Contêiner (o seu)
Tem um sistema operacional completo próprio?	Sim	Não
Tamanho (ordem de grandeza)	Geralmente vários GB	52.7MB
Tempo para iniciar	Geralmente dezenas de segundos a minutos	0,556 s (real)
O que compartilha com a máquina hospedeira?	Recursos físicos, como CPU, RAM e armazenamento, mediados pelo hipervisor.	Kernel do sistema operacional hospedeiro

Obs.: Evidência coletadas a partir dos comandos passados anteriormente:

- docker images projeto-devops
- time docker run --rm projeto-devops:v1 python -c "print('oi')"

3. Evidências: os dois prints do passo 9 (docker ps e curl), o nome e a tag da imagem, a porta mapeada e o número da linha do Dockerfile onde aparece a sua porta.





4. Linha do tempo: cole o resultado de git log --oneline -5 e escreva, ao lado de cada commit, quem fez e o que mudou.

(.venv) @GabrielFavaroo -> /workspaces/projeto-devops-dupla-11 (main) $ git log --oneline -5

c291cd3 (HEAD -> main, origin/main, origin/HEAD) Dockerfile da API

a23872d API Flask da dupla - 09/11/2026

ce2b6ec Merge pull request #1 from Edu1Sinico/variacao-grupo-onze

c79d8cd (origin/variacao-grupo-onze, variacao-grupo-onze) Aplica a variação da dupla

2535834 Fluxo que pública o site no Pages

5. Um problema e a solução: algo que travou a dupla e como foi resolvido.

Não tivemos problemas, a não ser o erro proposital de criar o container sem expor uma porta, que depois foi corrigido ao atribuir a porta necessária.

6. O que a dupla aprendeu, em três frases.

- Como criar um Dockerfile e um Dockerignore.
- Parâmetros que devem ser seguidos na imagem para que a aplicação esteja disponível.
- O processo de compilação de código e como funciona o Cache de certas camadas da aplicação.

Questionário:

Bloco 1. Identificação:

Respondemos este no bloco no tópico “Identificação da dupla 11”.

Bloco 2. Conceitos:

1. O que um contêiner compartilha com a máquina que o hospeda, e uma máquina virtual não?

a) Um sistema operacional completo, com o próprio kernel

b) O kernel do sistema operacional da máquina hospedeira

c) Um hardware emulado, só dele

d) Nada: o contêiner é totalmente isolado

2. Qual é a relação entre imagem e contêiner?

a) A imagem é o modelo pronto, e o contêiner é uma cópia dela em execução

b) A imagem é o contêiner parado, e não pode gerar outros

c) O contêiner é o modelo, e a imagem é a cópia em execução

d) São dois nomes para a mesma coisa

3. O que a linha `EXPOSE` do Dockerfile faz?

a) Abre a porta da API para a internet

b) Mapeia a porta do contêiner para a da máquina

c) Documenta a porta da API, sem abri-la sozinha

d) Inicia a API

4. No comando `docker run -d --name api -p 5007:5007 projeto-devops:v1`, o que faz o `-p 5007:5007`?

a) Define a senha do contêiner

b) Liga a porta 5007 da máquina à porta 5007 do contêiner

c) Escolhe a versão do Python

d) Limita o contêiner a 5007 MB de memória

5. Você mudou só a `MENSAGEM` no `app.py` e reconstruiu a imagem. Por que o `pip install` apareceu como `CACHED`?

a) Porque o Docker não refaz nenhum passo quando o código muda

b) Porque o `pip install` só roda na primeira vez da vida da máquina

c) Porque o Docker apagou o cache do `FROM`

d) Porque o `requirements.txt` não mudou e foi copiado antes do código, então essa camada foi reaproveitada

6. Para que serve o arquivo `.dockerignore`?

a) Evitar que arquivos desnecessários, como `.venv` e `.git`, sejam copiados para a imagem

b) Impedir que o contêiner acesse a internet

c) Esconder o Dockerfile do repositório

d) Guardar a senha do Docker Hub

Bloco 3. Evidências do seu repositório e do seu contêiner:

7. Link do repositório da dupla.

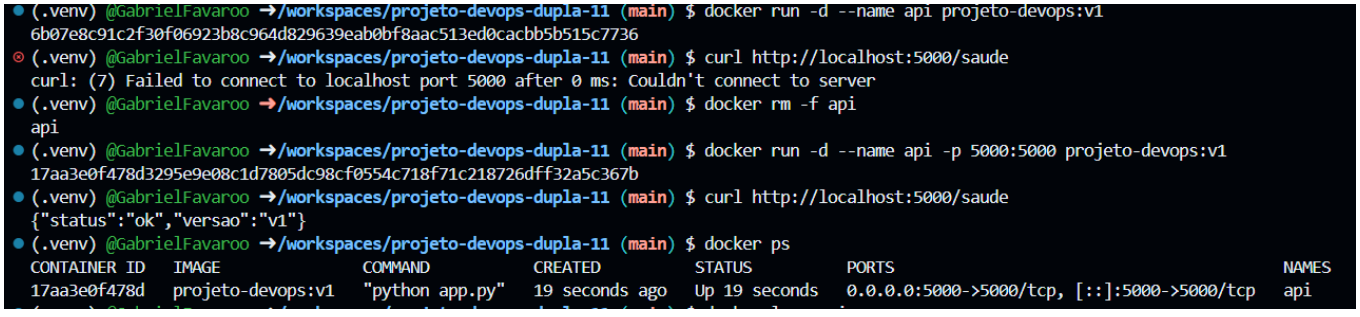
Repositório do Github: https://github.com/Edu1Sinico/projeto-devops-dupla-11

8. Os três valores da sua variação: NOME_DUPLA, MENSAGEM e PORTA.

NOME_DUPLA = "dupla-11"
MENSAGEM = "Ola, bem-vindo(a) a API da dupla 11!"
PORTA = 5000

9. Nome e tag da imagem que você construiu (coluna REPOSITORY e TAG do docker images).
As imagens construídas foram projeto-devops:v1 e projeto-devops:v2, sendo a primeira a versão inicial da aplicação e a segunda a versão utilizada para testar as alterações e o cache de camadas do Docker.

10. O comando docker run completo que você usou para subir o contêiner com a porta mapeada.



11. Em qual número de linha do Dockerfile aparece a sua porta, e qual é o valor escrito nela?
A porta está definida na linha 6 do Dockerfile, por meio da instrução EXPOSE 5000, que documenta a porta utilizada pela aplicação dentro do contêiner.

12. Qual é o valor da coluna SIZE da sua imagem e o tempo real que o time docker run mostrou?

- Valor do Size: 52.7MB (real)
- Tempo real: 0,556 s (real)

13. No build da versão v2 (passo 7), quais linhas do Dockerfile apareceram como CACHED?

Resultado:
=> CACHED [2/5] WORKDIR /app

=> CACHED [3/5] COPY app/requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir
=> CACHED [5/5] COPY app/ .

14. Rode git log --format="%h %an %s" -3 e escreva o código curto do commit que adicionou o Dockerfile, quem o fez e a mensagem.

```
@Edu1Sinico →/workspaces/projeto-devops-dupla-11 (main) $ git log --format="%h %an %s" -3
c291cd3 Gabriel Barbosa Dockerfile da API
a23872d Eduardo Sinico API Flask da dupla - 09/11/2026
ce2b6ec Eduardo Sinico Merge pull request #1 from Edu1Sinico/variacao-grupo-onze
```

Bloco 4. Só para o aluno A:

15. Passo 3: o que você previu que /saude e /aluno devolveriam, e o que devolveram de verdade? Responda em 2 a 3 linhas.

Previsão: A rota "/saude" retornará um código de status positivo (200) e a versão do sistema, ambos no formato de texto JSON. Já a rota "/aluno", retornará o nome da dupla e a mensagem definida no "app.py", também em formato JSON.

Resultado: Ao executar o comando "*curl http://localhost:5000/saude*", obtivemos o seguinte retorno em formato JSON:

{"status":"ok","versao":"v1"}

Repetindo o mesmo processo com o comando "*curl http://localhost:5000/aluno*", temos o seguinte retorno em formato JSON:

{"dupla":"dupla-11","mensagem":"Ola, bem-vindo(a) a API da dupla 11!"}

16. Passo 7: o que você previu que seria refeito no build ao mudar só a MENSAGEM, e o que realmente foi refeito? Responda em 2 a 3 linhas.

Minha previsão era que apenas a etapa de cópia do código da API fosse refeita, pois somente a variável MENSAGEM havia sido alterada, mantendo as dependências inalteradas.

Entretanto, durante o build da versão v2, todas as etapas apareceram como CACHED, indicando que o Docker conseguiu reutilizar os resultados de construções anteriores.

Bloco 5. Só para o aluno B:

17. Passo 4: liste os comandos que você precisou rodar para fazer a API funcionar do zero no seu Codespace e o que deu errado no caminho. Responda em 3 a 5 linhas.

- docker build -t projeto-devops:v1 **===> cria a imagem baseada nas instruções do dockerfile**
- docker run -d --name api projeto-devops:v1 **===> executa as instruções da imagem em um container**
- docker rm -f api **===> desliga o container para realizar alterações**
- docker run -d --name api -p 5000:5000 projeto-devops:v1 **===> executa novamente as instruções agora**

Atribuindo uma porta para acessar a aplicação

Ao executar o docker run sem atribuir a porta pelo -p PORTA:PORTA a aplicação subiu sem poder ser acessada e teve que ser desligada para depois adicionarmos a porta desejada, que fez que a aplicação ficasse acessível a requisições.

18. Passo 6: você previu que o curl funcionaria sem o -p? O que aconteceu, e por que isso acontece? Responda em 2 a 3 linhas.

Previsão: Acreditamos que o próprio Docker atribuiria uma porta padrão para rodar a nossa aplicação.

Resultado: Entretanto, após executar os primeiros comandos, o resultado foi o seguinte:

```
curl http://localhost:5000/saude
curl: (7) Failed to connect to localhost port 5000 after 1 ms: Couldn't connect to server
```

Resposta: Isso ocorreu pois não definimos em qual porta o sistema irá rodar, por conta disso, ao tentar executar o comando, o Docker não consegue identificar qual porta o sistema está e acaba retornando um erro.

Podemos identificar se o conteúdo alocado no container com o comando "*Docker ps*", assim, ele retorna várias informações, inclusive a porta

Bloco 6. Perguntas abertas:

19. "Funciona na minha máquina." Explique como o contêiner resolve esse problema, usando o que aconteceu com o aluno B no passo 4.

O problema de "*Funciona na minha máquina*" normalmente ocorre por diferenças de sistema operacional, versão software necessário para executar a aplicação, etc. Ao criar uma Docker Image, ela atribui todas as regras de como será o ambiente de execução, e todos os usuários utilizaram um ambiente exatamente como esse que é o container, eliminando divergências de versão e outras decorrentes de uma execução sem o Docker.

20. Qual é a diferença entre um contêiner e uma máquina virtual? Use pelo menos um número que você mediu no passo 8 para sustentar a resposta.

A máquina virtual é responsável por emular um sistema operacional, utilizando da parte dos recursos físicos de nosso computador para toda a operação. Já o contêiner, ele não possui um sistema operacional próprio, mas compartilha diretamente do kernel, no qual, utiliza dos recursos para isolar a aplicação e suas dependências.

Há também a diferença de tamanho e velocidade, no qual, a VM costuma utilizar vários GB e possui um tempo de dezenas de segundas a minutos. O contêiner apresenta ser menor no espaço e tempo, como no nosso exemplo do projeto, nosso contêiner possui 52.7MB e um tempo de inicialização de 0.556.